FINAL REVIEW

TITLE: DiCE-Flow

GUIDE NAME: SHILPA S **GROUP NO: 20**

GROUP MEMBERS:
SAURAV SREEJITH
SHREYA PADMAKUMAR
S MURUGAN

Overview

- Introduction
- Problem Definition
- Objective Of The Work
- Feasibility Analysis & Tool Specifications
- Methodology
- Results and Discussions
- Identification of SDGs
- Conclusion
- References

Introduction

- Biological systems can be modeled as **complex interaction networks** where:
  - **Nodes** represent genes or proteins
  - **Edges** represent biological interactions such as signaling or protein binding
- Understanding how diseases alter these networks helps identify **critical disease-associated genes**.
- Traditional gene analysis focuses mainly on **expression changes**, but many disease drivers emerge due to **network topology changes**.
- To address this, our project integrates:
  - Gene expression data
  - Biological interaction networks
  - Graph algorithms for centrality analysis

The Biological Gap: Overlooking Silent Drivers

- **Current Limit:** Traditional cancer analysis, primarily Differential Expression Analysis (DEA) only counts gene abundance.
- **The Blind Spot:** It misses "silent" driver genes that rewire connections without changing expression levels.
- **The Cost:** Critical therapeutic targets (Hub Genes) are overlooked because they fail standard statistical cutoffs (e.g., limma, edgeR).

The Computational Gap: Centrality Bottlenecks

- **The Need:** Finding silent drivers requires **Betweenness Centrality**, necessitating All-Pairs Shortest Path calculations.

- **The Bottleneck:** On large bio-networks (>20k nodes), standard Dijkstra is too slow.
  $(O(V \cdot E \log V))$

- **The Solution:** A specialized, high-performance Single Source Shortest Path (SSSP) engine is required to handle massive biological graphs.

Objective of the Work

1

Implement DiCE Pipeline

To identify cancer-driving genes based on network topology rather than solely on gene expression levels. This involves developing the Differential Centrality-Ensemble (DiCE) pipeline.

2

Engineer High-Performance Graph Engine

To accelerate centrality computations using the **Duan 2025 SSSP algorithm** heuristics, specifically Recursive Shell Decomposition, implemented in a high-performance C++ Graph Engine.

Feasibility Analysis

Technical Feasibility:

The project leverages a robust foundation:

- C++ for high-performance core computations
- Python for data processing and analysis.

This combination ensures stability and efficiency.

Data Availability:

Utilizes publicly accessible, high-quality datasets.

- TCGA (The Cancer Genome Atlas) - provides extensive gene expression data
- BioGRID and KEGG offer comprehensive Protein-Protein Interaction (PPI) networks.

Tool Specification: Core Technologies

- **Languages:**

Python: Data pipeline, Analysis, & FastAPI Backend.
C++: Core high-performance graph engine.
TypeScript (React): Interactive Frontend & UI logic.

- **Libraries:**

Backend: FastAPI, NetworkX, Pandas, NumPy.
Frontend: Vite, Tailwind CSS.

- **Visualization:**

Interactive: Cytoscape.js (Network rendering).
Static: Matplotlib & Seaborn (Report generation).

Methodology: 5-Phase Pipeline Architecture

01

Network Construction

KEGG + BioGRID database merge

02

Expression Weighting

Pearson correlation distance transformation

03

C++ Centrality

Brandes algorithm + Eigenvector computation

04

Differential Ranking

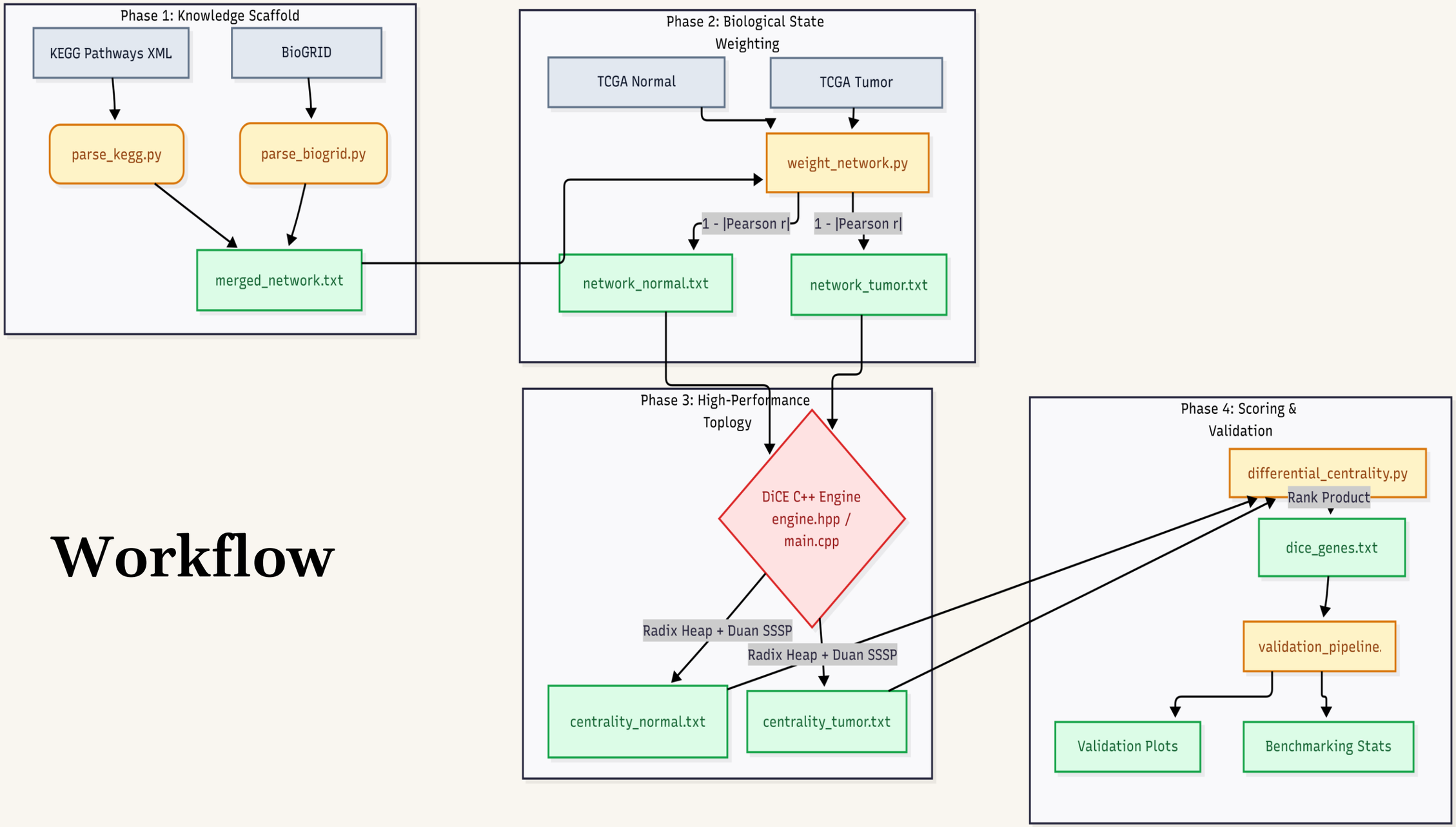
Ensemble score product calculation

05

Validation

Hypergeometric test + Permutation analysis

Workflow


Phase 1: Knowledge Scaffold
KEGG Pathways XML
BioGRID
parse_kegg.py
parse_biogrid.py
merged_network.txt
Phase 2: Biological State Weighting
TCGA Normal
TCGA Tumor
weight_network.py
1 - |Pearson r|
1 - |Pearson r|
network_normal.txt
network_tumor.txt
Phase 3: High-Performance Toplogy
DiCE C++ Engine
engine.hpp / main.cpp
Radix Heap + Duan SSSP
Radix Heap + Duan SSSP
centrality_normal.txt
centrality_tumor.txt
Phase 4: Scoring & Validation
differential_centrality.py
Rank Product
dice_genes.txt
validation_pipeline.
Validation Plots
Benchmarking Stats

Module 1: Network Construction
KEGG + BioGRID Fusion Strategy

KEGG Edge Directionality

Signaling pathway direction preserved as directed edges (A → B)

BioGRID Bidirectionalization

Physical interactions converted to bidirectional edges (A ↔ B)

KEGG Priority Rule

BioGRID edge skipped if identical KEGG interaction exists

Duplicate Removal

Self-loops and redundant edges dropped (final: 1,882,598 edges)

Module 2: Expression Weighting
Condition-Specific Edge Distance Transformation

$$Weight(A,B) = 1 - |\rho(expr_A, expr_B)|$$

This transformation converts Pearson correlation coefficients ($\rho \in [-1,1]$) into non-negative edge weights ($w \in [0,2]$), where **high correlation** produces low weights (short functional distance) and **low correlation** produces high weights (decoupled in condition).

Module 3: C++ Centrality Engine

- **Differential Centrality Calculation**

Compute the absolute difference in centrality scores between tumor and normal samples:
|Δ Centrality| = |Tumor - Normal|.
This highlights genes with significant topological changes.

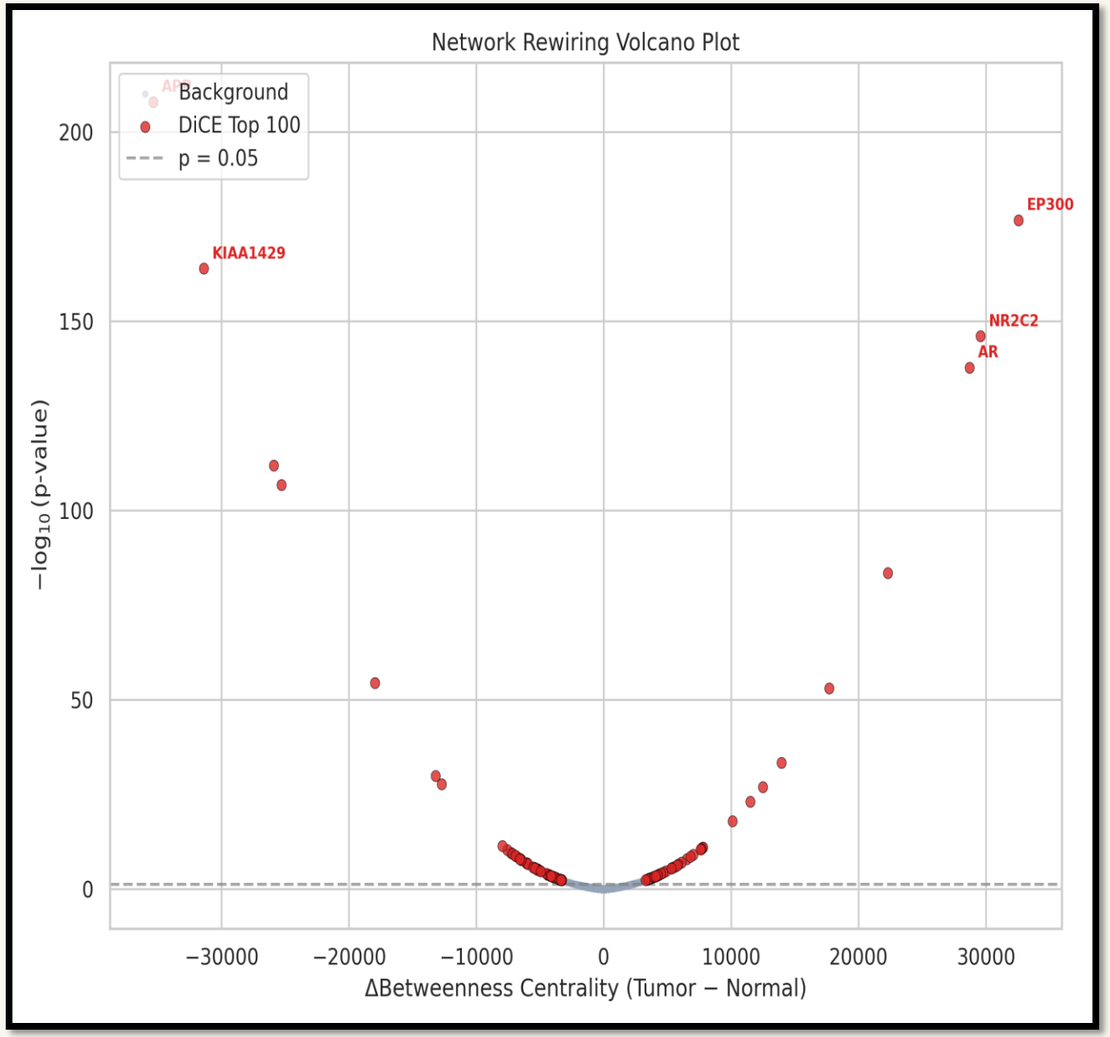
- **Rank Product Combination**

To identify robust driver candidates, we combine the ranks of both Betweenness and Eigenvector centrality changes using a rank product approach, ensuring a comprehensive assessment.

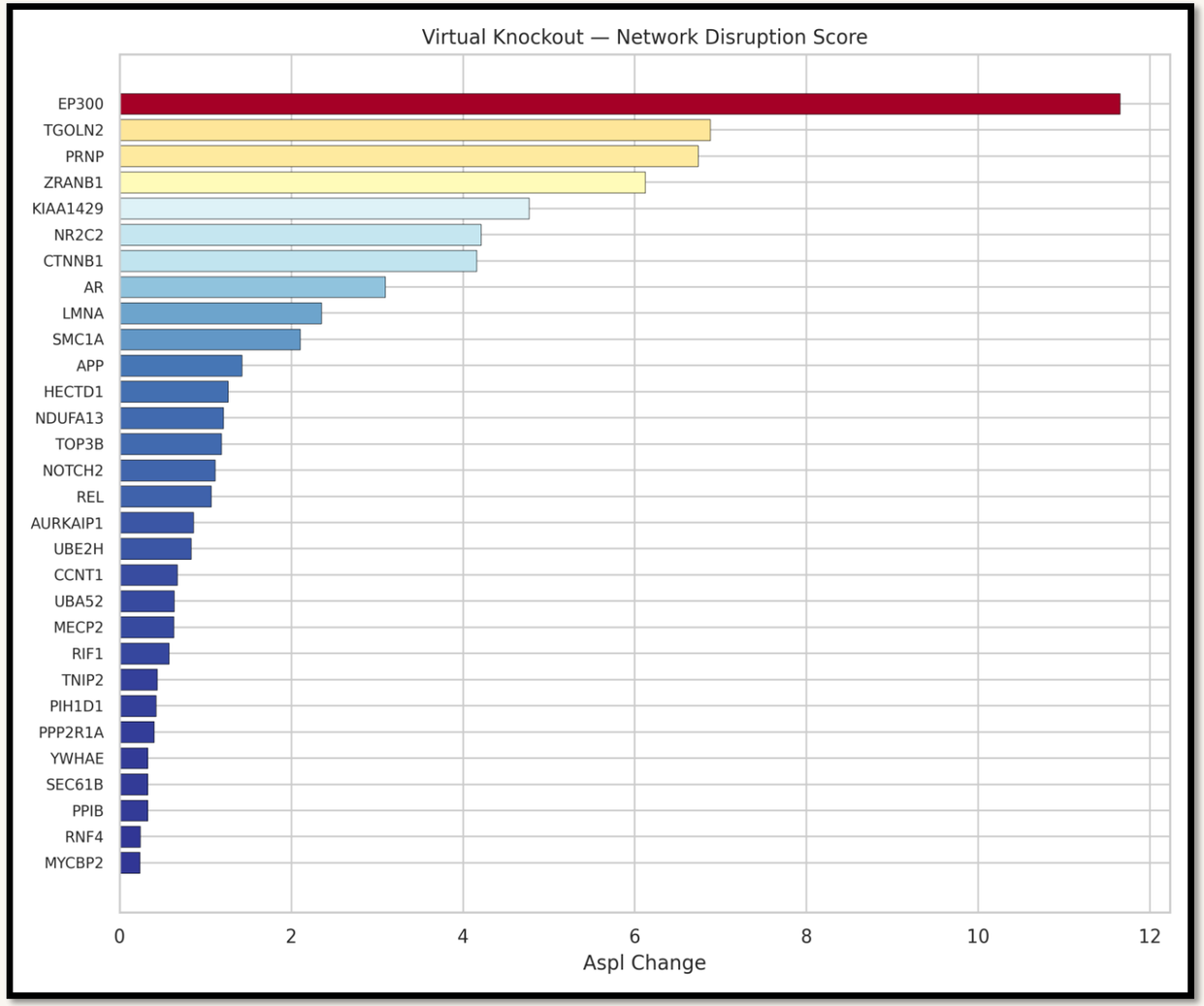
- **Virtual Knockout Simulation**

A "Virtual Knockout" experiment simulates the removal of top-ranked candidate genes. This allows us to calculate a "Vitality" score, quantifying the network disruption caused by each gene's absence and assessing its critical role.

Results and Discussions

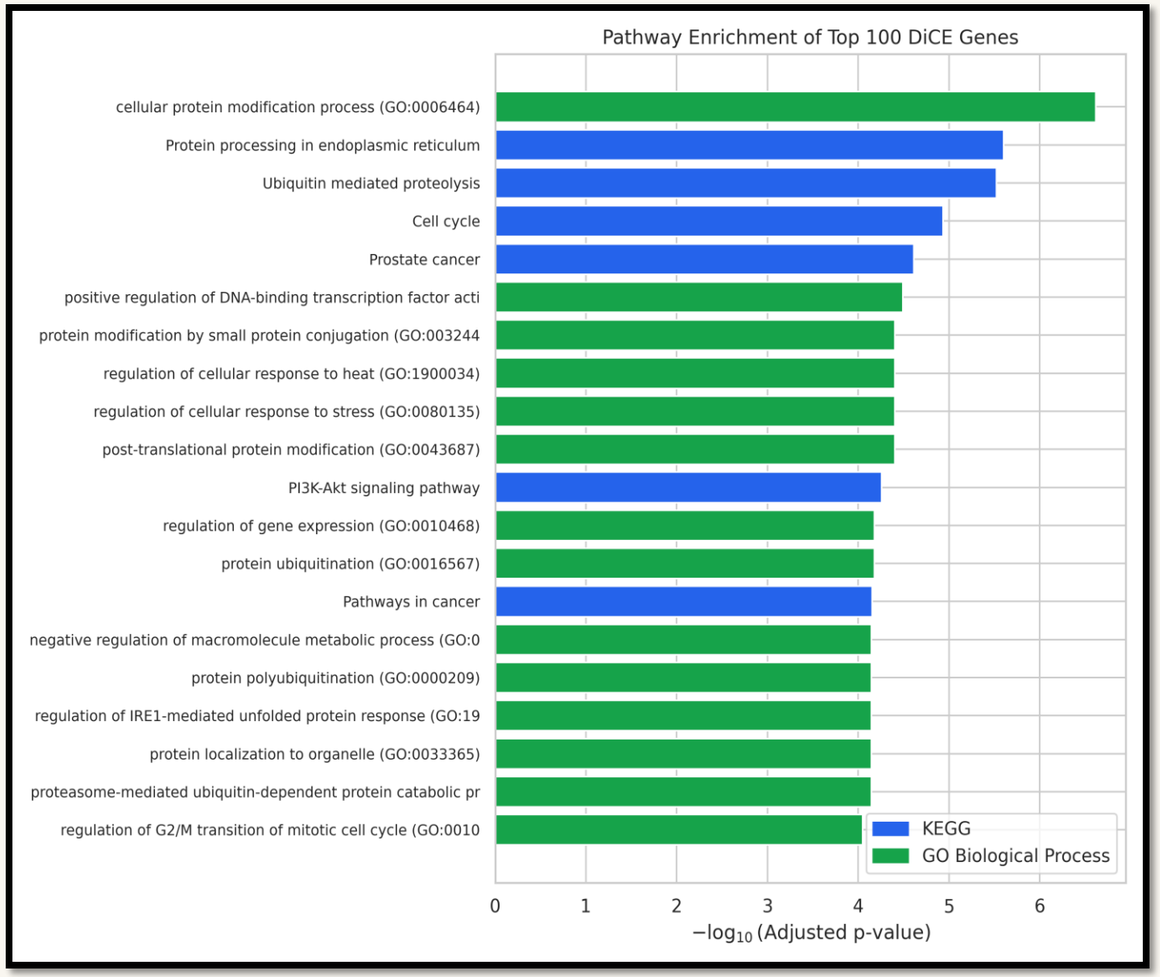


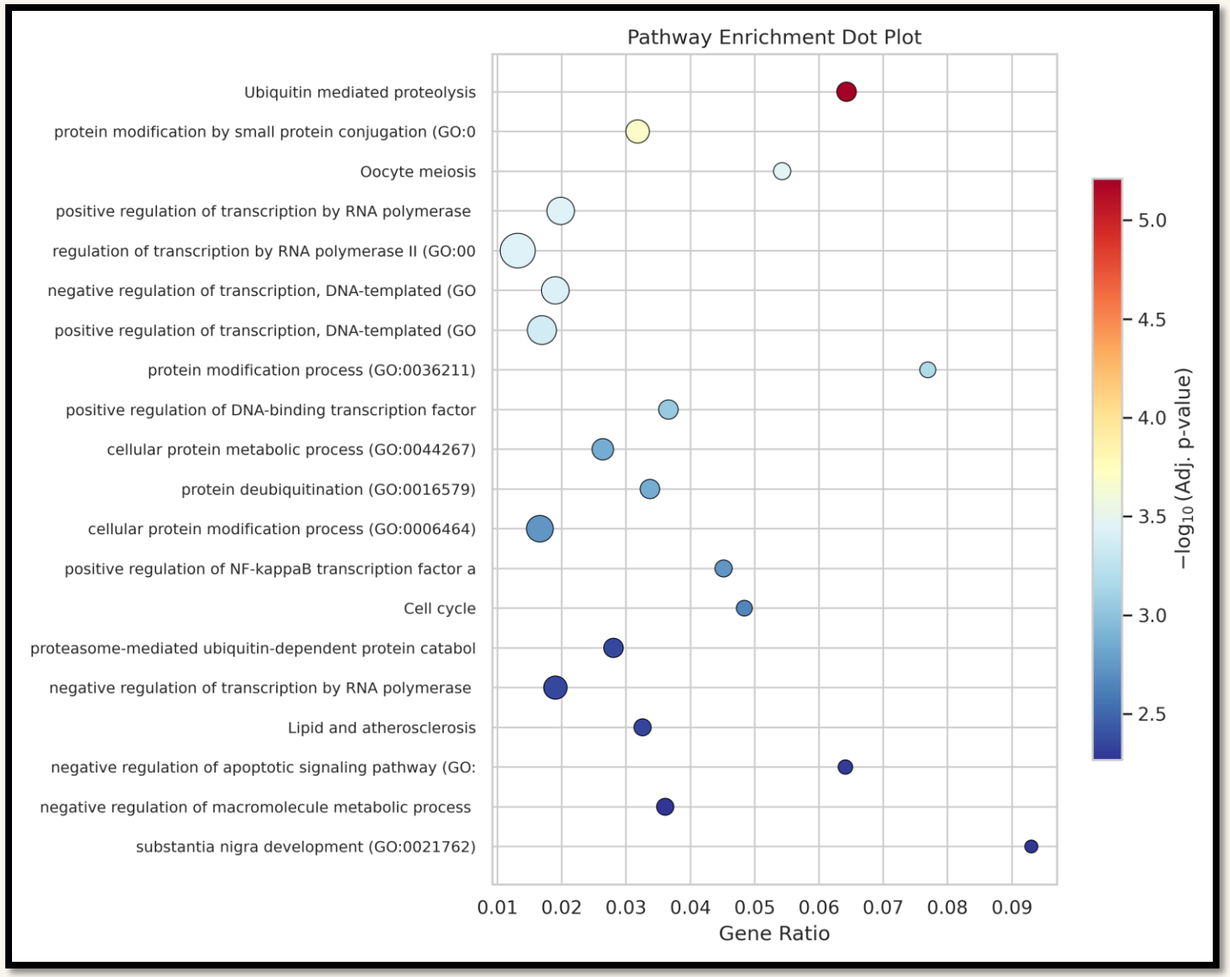
Rewiring volcano



Virtual Knockout

Results and Discussions

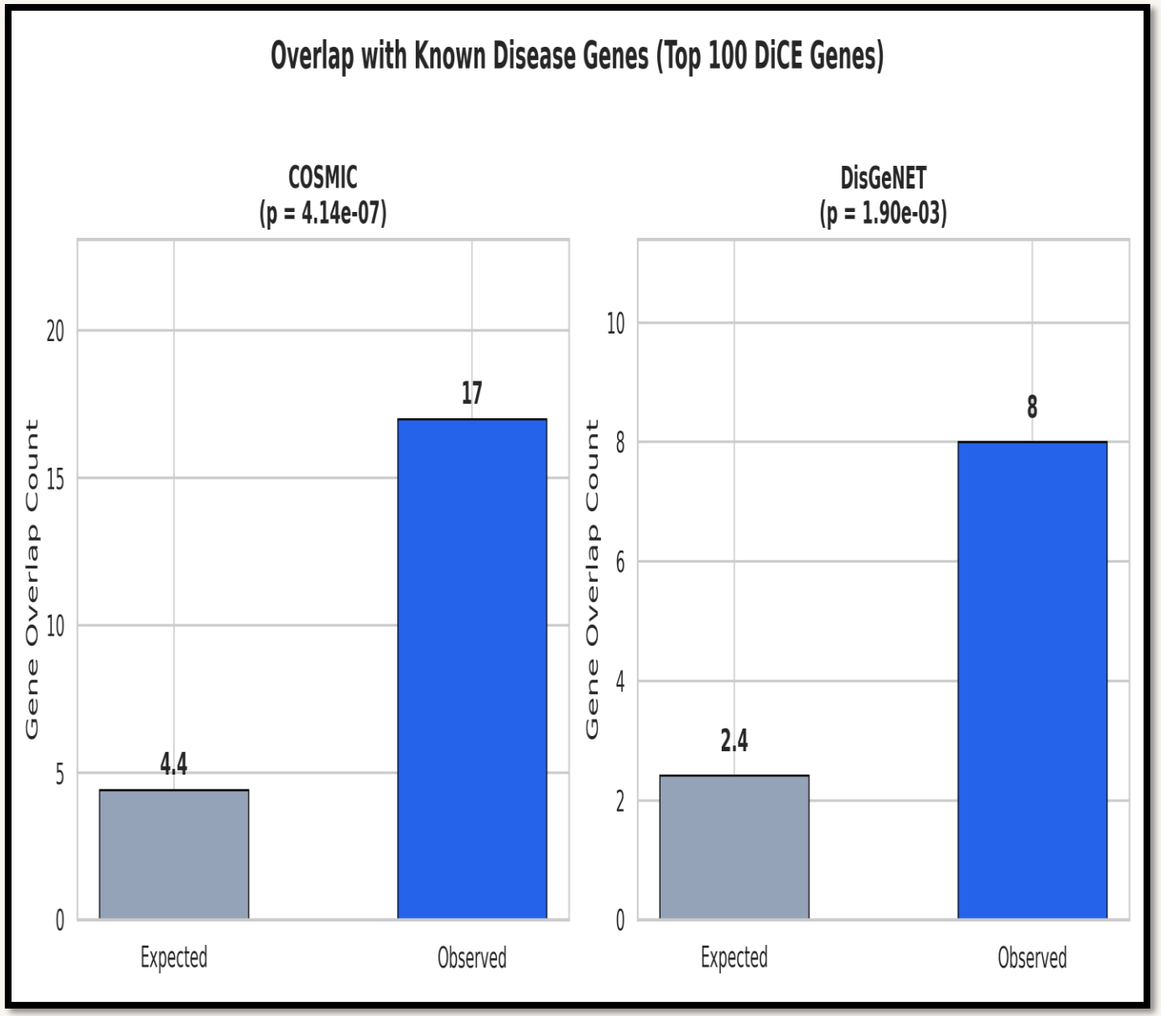




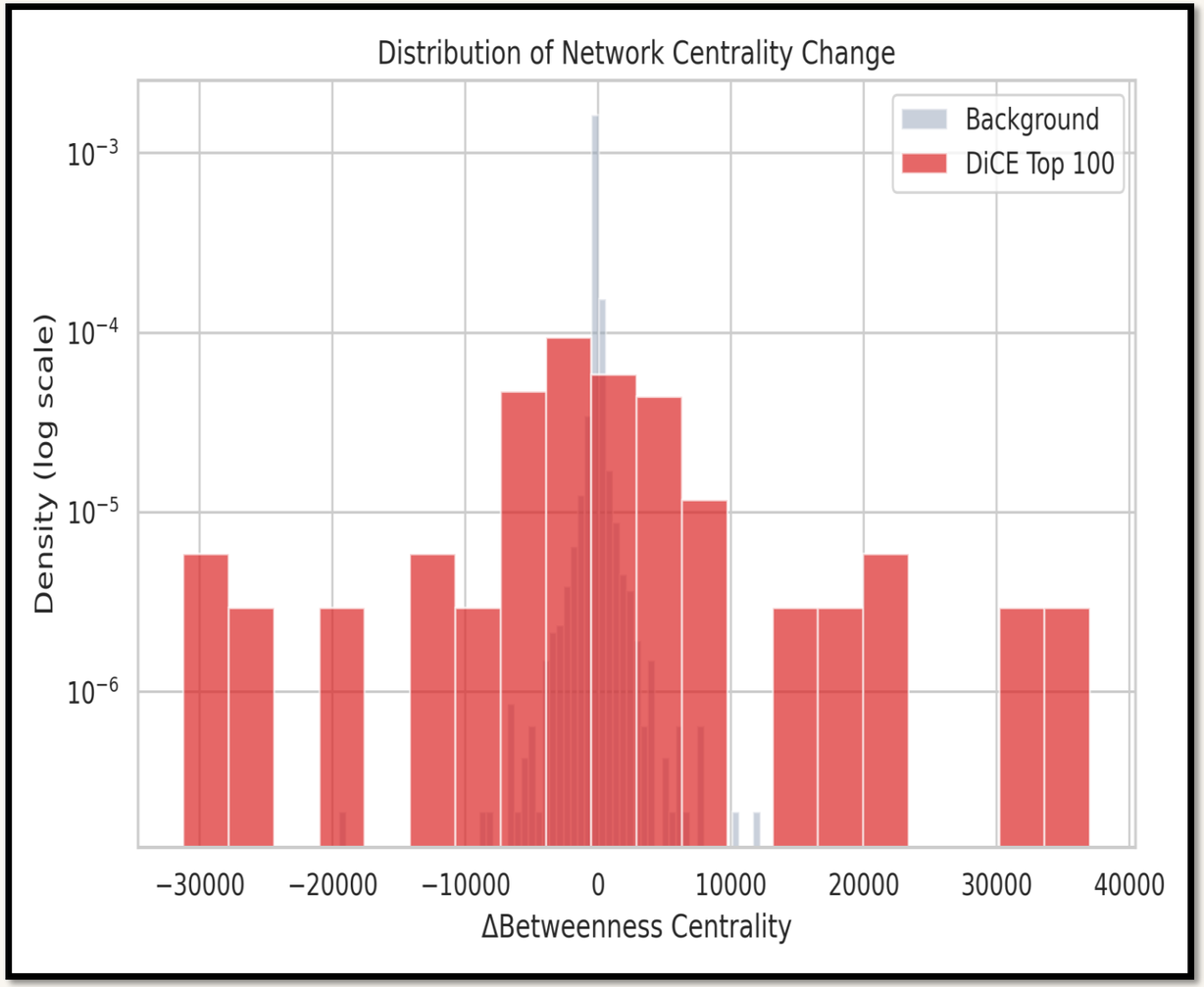
Pathway Enrichment Boxplot

Pathway Enrichment Dotplot

Results and Discussions

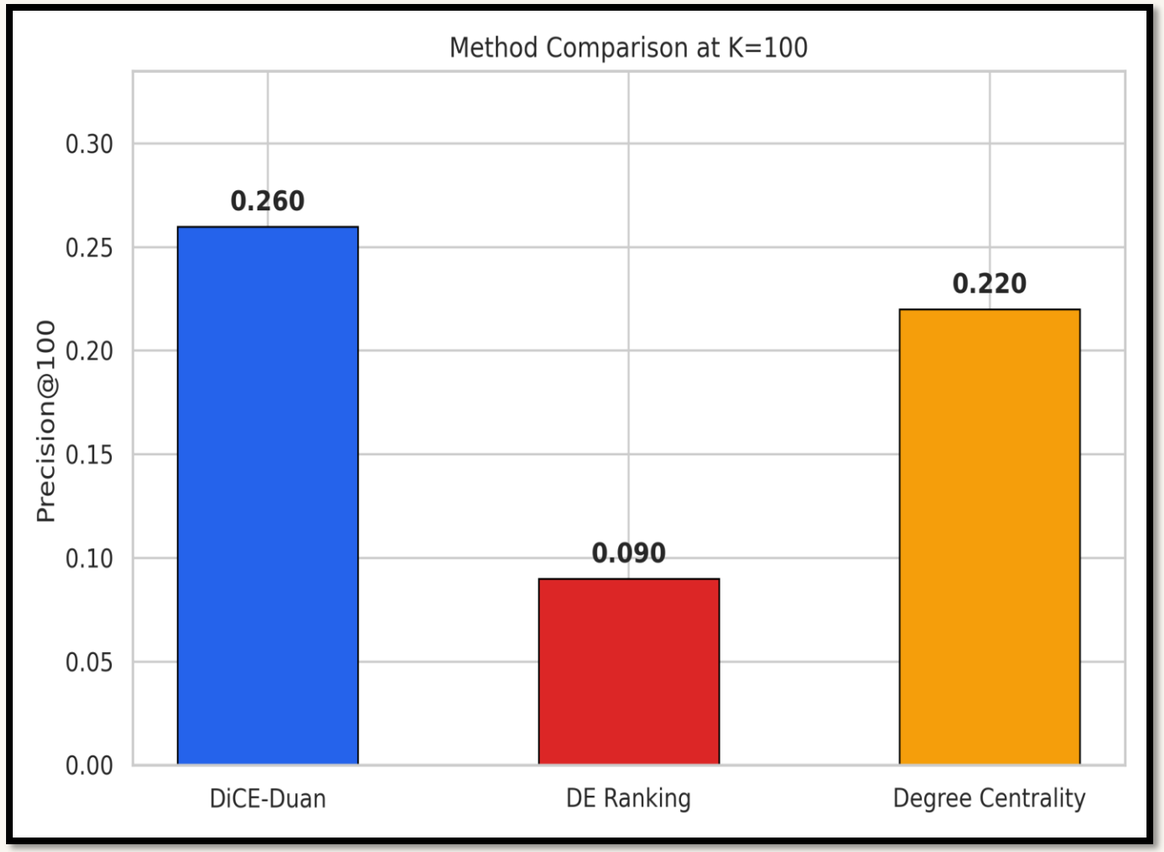


Hypergeometric Test

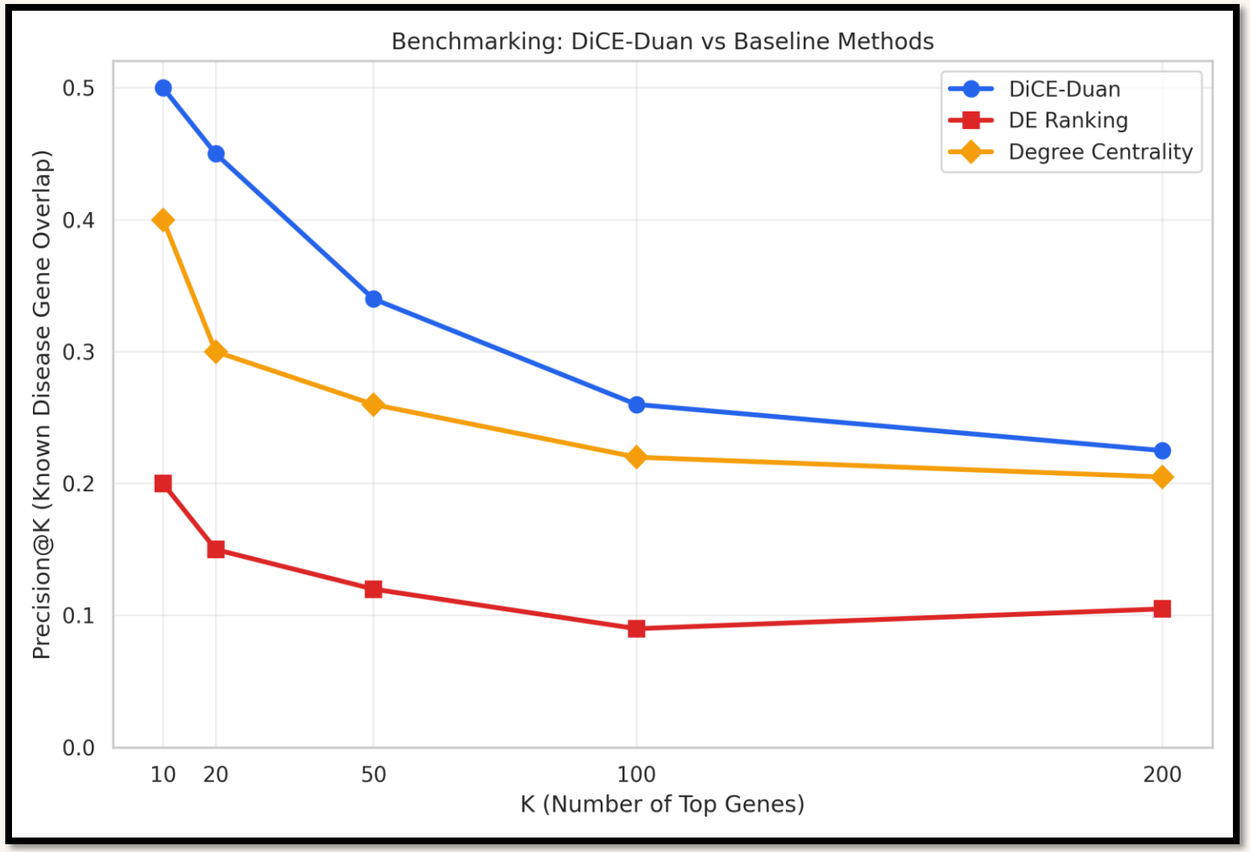


Rewiring Histogram

Results and Discussions



Benchmark bar chart



Benchmarking Precision@K curve

Sustainable Development Goals

Global Impact Alignment

SDG 3: Good Health and Well-Being

DiCE directly advances cancer genomics by identifying EP300, AR, and ZRANB1 as network hubs, providing computational targets for precision oncology. Early-stage bioinformatics tools reduce experimental drug target validation costs.

SDG 17: Partnerships for the Goals

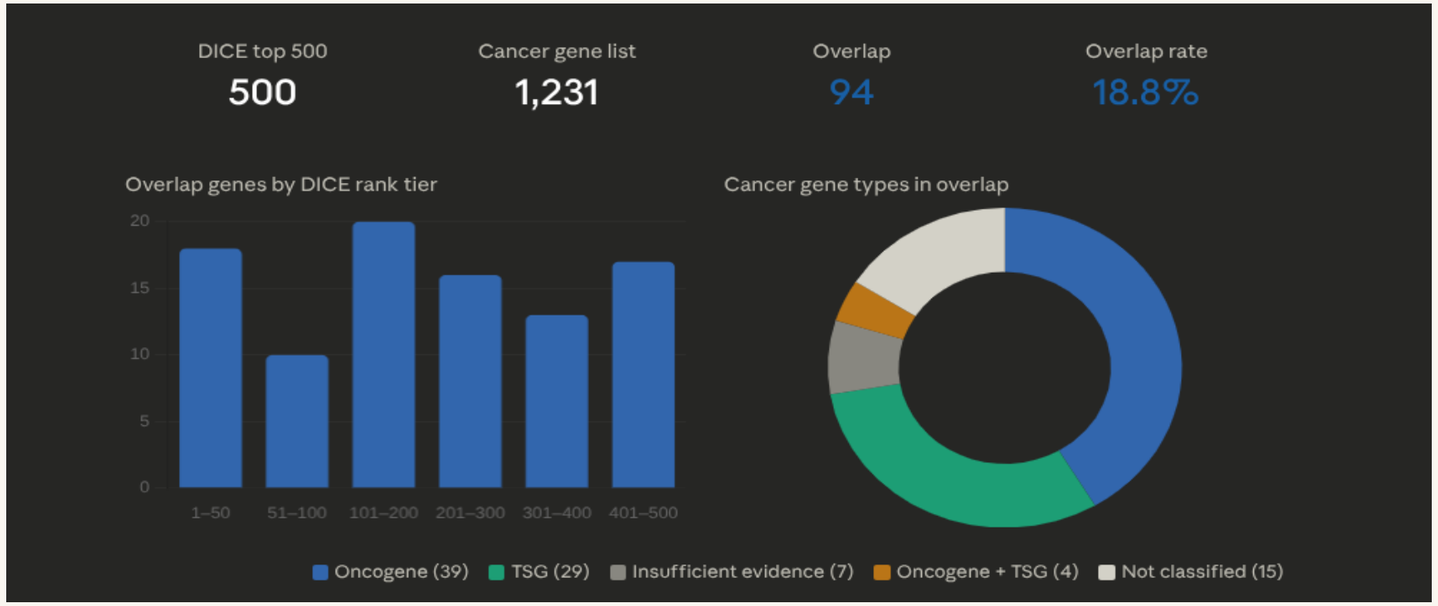
Open-source C++ + Python pipeline represents research infrastructure innovation — combining recursive SSSP, chunked memory processing, and REST API serving into reproducible bioinformatics stack accessible to any research group.

Conclusion: What DiCE-Flow Delivers

DiCE identifies cancer-associated genes by measuring network centrality rewiring between normal and tumor states — **without using disease labels as input.** KEGG + BioGRID fusion avoids co-expression circularity present in STRING-based approaches, providing an independent biological prior for edge weighting.

Open-Source Pipeline

Fully reproducible, scalable to any cancer type, extensible to single-cell or multi-omic data



Team Contributions

Task Distribution Among Group Members

Saurav Sreejith

C++ Engine & Centrality Computation

- `engine.hpp` — Radix heap, recursive shell decomposition
- Pearson co-relation and Betweenness centrality — Brandes algorithm
- server.py (FastAPI)
- main.cpp — Virtual knockout ASPL analysis

Shreya Padmakumar

Analysis, Validation & Serving Layer

- `weight_network.py` — vectorized Pearson, chunked OOM fix
- Eigenvector centrality — weighted power iteration, convergence
- Validation suite — hypergeometric, Precision@K

S Murugan

Data Engineering & Network Construction

- `parse_kegg.py` — KGML XML parsing, multi-gene expansion
- `parse_biogrid.py` — 2.8M row chunked processing
- `merge_networks.py` — KEGG-priority fusion logic
- `pre_filter.py` — Information gain IG filter
- differential_centralty.py — ensemble rank product scoring

References

1. Pashaei, E., et al. (2025). "DiCE: differential centrality-ensemble analysis based on gene expression profiles and protein–protein interaction network." *Nucleic Acids Research*, 53(13).
2. Duan, R., et al. (2025). "Breaking the Sorting Barrier for Directed Single-Source Shortest Paths." *In Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC).* (Recipient of Best Paper Award).

THANK YOU